

Mini SIEM Capstone Project – Week 9 Progress Report

Trent Carter, COMP 460

Project Overview

The goal of this project is to design and implement a simplified Security Information and Event Management (SIEM) system capable of ingesting log data, normalizing events into a unified schema, storing them in a database, and performing detection and correlation across multiple data sources. The system has progressed from basic log ingestion to a fully functional detection engine capable of identifying and validating suspicious behavior using real-world cybersecurity data.

Summary of Week 9 Work

During Week 9, the project transitioned from multi-source correlation to a fully implemented detection and validation system. Building on the LANL dataset integration from Week 8, a behavior-based detection engine was developed to identify suspicious activity patterns across authentication, process, network, and DNS logs.

Unlike earlier stages of the project that relied on predefined event types, this phase focused on analyzing raw system activity and detecting patterns that resemble attacker behavior. Detection results were then validated using the LANL red team dataset, which provides ground truth indicators of attacker movement within the network.

Implementation Details

1. Detection Engine Development

A detection engine (`lanl_detection.py`) was created to analyze normalized and correlated events stored in the SQLite database. The engine applies multiple rule-based detection strategies to identify suspicious activity across systems.

Detection rules implemented include:

- **Time-Based Authentication and Process Correlation**
Identifies systems where authentication events are closely followed by process execution within a defined time window.
- **Time-Based Process and Network Correlation**
Detects processes that initiate network connections shortly after execution, which may indicate outbound communication or lateral movement.
- **Multi-Stage Behavior Chain Detection**
Detects sequences of activity involving authentication, process execution, and network communication occurring within a short time window on the same system.
- **Red Team DNS Correlation**
Identifies DNS activity associated with systems involved in known attacker behavior.

2. Time-Based Correlation

Detection rules were enhanced with time-based filtering to improve accuracy. Events are only considered related if they occur within a defined threshold (500 time units), ensuring that correlations reflect realistic sequences of activity rather than unrelated events.

This addition significantly reduces noise and improves the reliability of detection results.

3. Alert Structuring and Severity Classification

Each detection generates a structured alert containing:

- unique alert ID
- rule name
- severity level (low, medium, high, critical)
- affected systems (source and destination)
- associated user
- descriptive message
- metadata including timestamps and event relationships

Severity levels are determined based on:

- whether the detected activity overlaps with known red team systems
- the complexity of the behavior chain (multi-stage vs. single-stage activity)

4. Detection Results and Validation

Detection Engine Output:

```
=== LANL Improved Detection Engine v2 ===  
Time window threshold: 500  
Limit per rule: 5000  
  
Total structured alerts generated: 16732  
Redteam-validated alerts: 12445  
Validation rate: 74.38%  
  
Alerts by rule:  
  timebound_auth_process_chain: 1732  
  timebound_process_network_chain: 5000  
  auth_process_network_chain: 5000  
  redteam_dns_overlap: 5000  
  
Alerts by severity:  
  high: 7445  
  low: 4287  
  critical: 5000
```

Detection results:

- Total structured alerts generated: 16,732
- Redteam-validated alerts: 12,445
- Validation rate: 74.38%

These results indicate that a majority of generated alerts are associated with systems involved in known attacker activity, demonstrating the effectiveness of the detection logic.

5. Alert Storage and Persistence

All generated alerts are persisted for analysis and future visualization:

- Exported to JSONL file (data/alerts/lanl_alerts.jsonl)
- Inserted into the SQLite alerts table

This ensures that detection results can be queried, analyzed, and displayed in later stages of the project.

6. Database Verification

Alert Database Verification:

```
>>> import sqlite3
>>>
>>> conn = sqlite3.connect("data/mini_siem.db")
>>> cursor = conn.cursor()
>>>
>>> cursor.execute("""
... SELECT COUNT(*)
... FROM alerts
... WHERE source = 'lanl_detection';
... """)
<sqlite3.Cursor object at 0x000001B63F04B040>
>>>
>>> print("Total LANL alerts in DB:", cursor.fetchone()[0])
Total LANL alerts in DB: 16732
```

Verification confirmed that all generated alerts (16,732) were successfully stored, ensuring consistency between detection output and persistent storage.

Key Takeaways

The most significant achievement of Week 9 is the successful implementation of a behavior-based detection engine capable of analyzing raw system activity across multiple log sources. The system now detects patterns that resemble attacker behavior rather than relying on predefined event labels.

This approach more accurately reflects real-world SIEM systems, where analysts must infer malicious activity based on event relationships and timing.

Additionally, the use of red team data for validation provides measurable evidence of detection effectiveness, demonstrating that the system is capable of identifying activity associated with known attacks.

Challenges Encountered

- Designing detection rules for unlabeled datasets
- Managing large volumes of correlated data
- Implementing time-based filtering to improve detection accuracy
- Structuring alerts for clarity and usability
- Validating detection results using indirect ground truth data

All challenges were successfully addressed while maintaining system performance and scalability.

Next Steps (Week 10 Plan)

The final phase of the project will focus on:

- Developing a user interface/dashboard for visualizing alerts
- Displaying detection results, severity levels, and correlations
- Improving usability and interaction with the SIEM system
- Final system testing and refinement
- Preparing final presentation and documentation